



**UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS**

Vicerrectoría
Oficina de Extensión

Académica

Implementación y Configuración de SSO con Keycloak para Aplicaciones Frontend

Ing. Juan David Acosta Rincón

2026



Contenido

1. Información General	4
2. Objetivo	4
3. Alcance	4
4. Arquitectura Implementada	5
5. Configuración del Entorno	5
5.1 Configuración del Contenedor	5
5.2 Configuración del Proxy Reverso	6
6. Configuración en Keycloak	6
6.1 Creación del Realm	6
6.2 Creación de Usuarios	8
7. Configuración del Cliente	9
7.1 Tipo de Cliente	9
8. Configuración de Seguridad del Cliente	11
8.1 Client Authentication	11
8.2 Authorization Services	11
9. Flujos de Autenticación (OAuth 2.0 / OIDC)	11
9.1 Standard Flow (Authorization Code Flow)	11
9.2 Implicit Flow	12
9.3 Direct Access Grants	12
9.4 Service Account Roles	12
9.5 Token Exchange, Device Flow y CIBA	12
10. PKCE (Proof Key for Code Exchange)	12



UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

Vicerrectoría
Oficina de Extensión

Académica

11.	DPoP Bound Tokens	13
12.	Gestión de Roles.....	13
13.	Implementación en Frontend.....	15
14.	Seguridad Implementada	15
15.	Resultados.....	16
16.	Conclusiones.....	16



1. Información General

Dependencia: Oficina de Extensión

Sistema: Servicio de Autenticación Centralizada (SSO)

Tecnología Principal: Keycloak

Tipo de Aplicación Integrada: SPA (Single Page Application)

Entorno: Producción con HTTPS y Proxy Reverso

Responsable: Juan David Acosta Rincon

Fecha: 05 – 03 – 2026

2. Objetivo

Documentar la configuración técnica implementada para la integración del sistema de autenticación centralizada (SSO) mediante Keycloak en aplicaciones frontend, garantizando:

- Autenticación segura bajo protocolo HTTPS.
- Funcionamiento correcto detrás de proxy reverso.
- Implementación adecuada de OAuth 2.0 / OpenID Connect.
- Gestión de roles y permisos desde el cliente.
- Cumplimiento de buenas prácticas de seguridad para aplicaciones SPA.

3. Alcance

El presente informe cubre:

- Configuración del contenedor Docker de Keycloak.
- Configuración del proxy reverso (Nginx).
- Creación de Realm (Reino).
- Creación de usuarios.
- Creación y configuración del cliente.
- Configuración de flujos OAuth.



- Implementación de middleware en frontend.
- Gestión de roles y permisos desde la aplicación.

4. Arquitectura Implementada

La arquitectura implementada se compone de:

1. Contenedor Docker ejecutando Keycloak.
2. Proxy reverso Nginx.
3. Entorno HTTPS obligatorio.
4. Aplicación frontend (Vite / JavaScript).
5. Comunicación mediante protocolo OpenID Connect (OIDC).

El servicio se encuentra expuesto bajo una ruta específica detrás de proxy, asegurando que todas las URLs generadas por el servidor utilicen HTTPS.

5. Configuración del Entorno

5.1 Configuración del Contenedor

Se realizaron configuraciones críticas para permitir que Keycloak opere correctamente detrás de proxy y bajo HTTPS:

- Activación de modo proxy (`KC_PROXY=edge`).
- Definición de hostname público.
- Definición de ruta relativa (`KC_HTTP_RELATIVE_PATH`).
- Desactivación de validaciones estrictas de hostname.
- Forzado de generación de URLs HTTPS.
- Configuración de cabeceras `x-forwarded`.

Estas configuraciones garantizan:

- Correcto manejo de redirecciones.
- Emisión adecuada de tokens.



- No generación de errores 401 por inconsistencias de dominio.
- Funcionamiento correcto del login bajo proxy.

5.2 Configuración del Proxy Reverso

Se configuró Nginx con:

- Reenvío de cabeceras X-Forwarded-*.
- Soporte para WebSockets.
- Manejo de cookies con reescritura de dominio.
- Eliminación de headers que interfieran con seguridad.
- Cacheo de recursos estáticos.

Esto asegura:

- Integración correcta entre proxy y Keycloak.
- Persistencia adecuada de sesión.
- Compatibilidad con SPA modernas.

6. Configuración en Keycloak

6.1 Creación del Realm

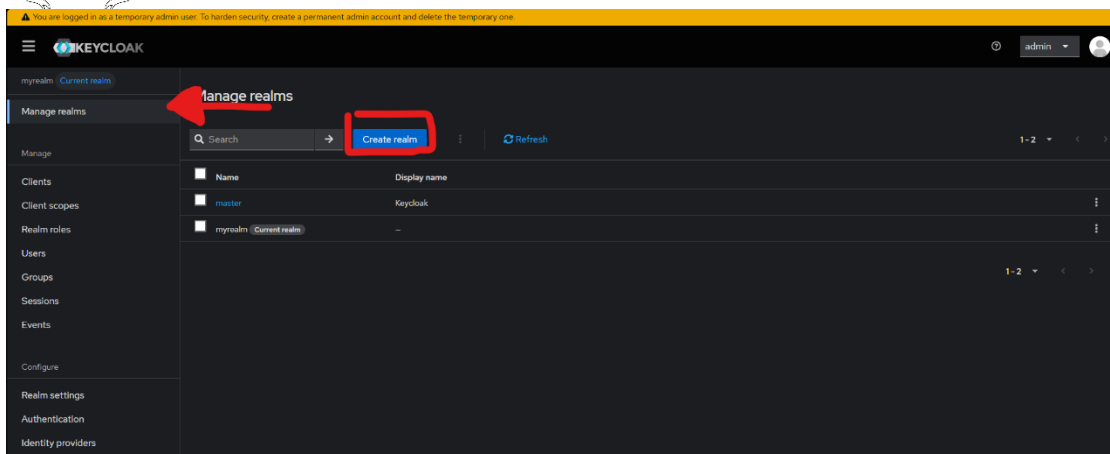
Se creó un Realm específico para el sistema, aislando la configuración de autenticación. Para esto primero se accede al panel de reinos/workspace



UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

Vicerrectoría
Oficina de Extensión

Académica



Posteriormente se le debe dar un nombre al reino/workspace que se quiera crear

Create realm

A realm manages a set of users, credentials, roles, and groups. A user belongs to and logs into a realm. Realms are isolated from one another and can only manage and authenticate the users that they control.

Resource file

Drag a file here or browse to upload Browse... Clear

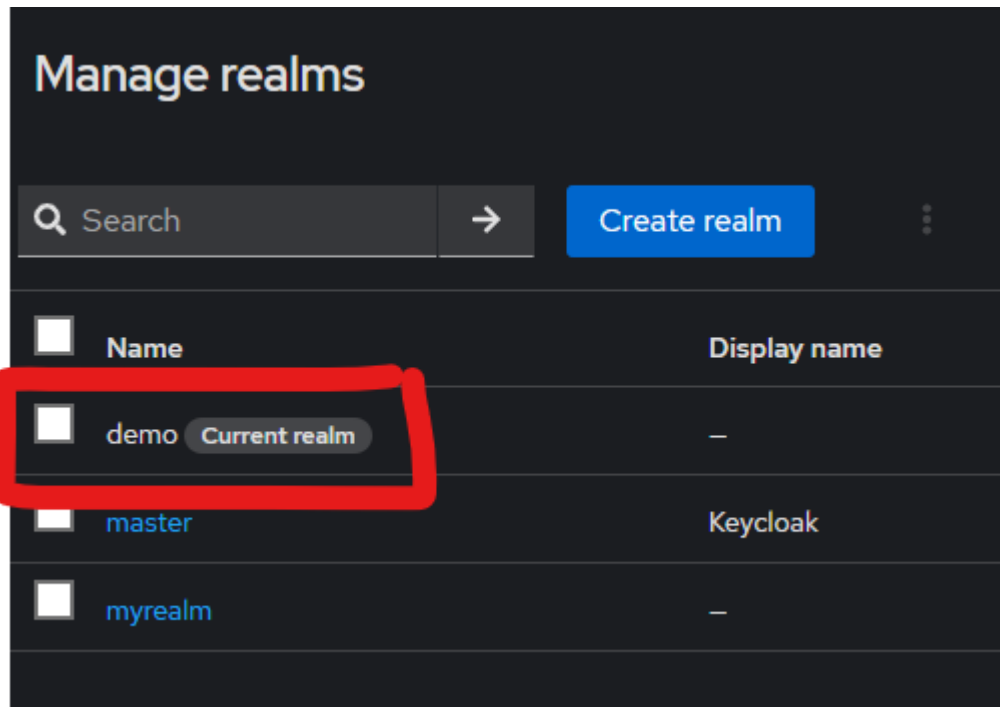
Upload a JSON file

Realm name *

Enabled ☒ Off

Create Cancel

Y por ultimo se debe verificar si se esta en el reino/workspace correcto, esto se logra gracias a que el sistema indica con una etiqueta de “Current real” en donde se esta trabajando.

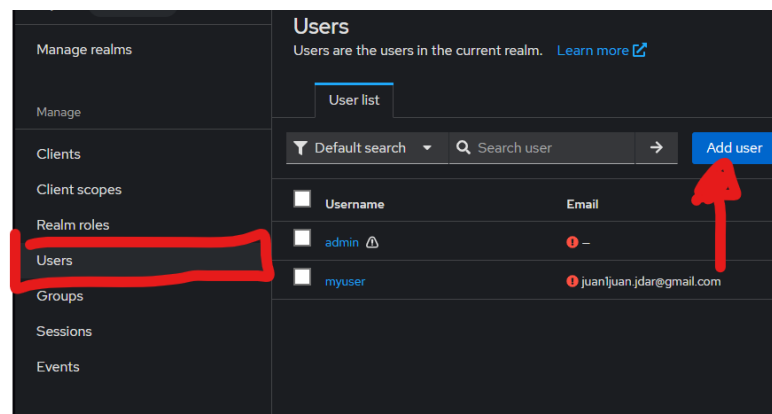


6.2 Creación de Usuarios

Se crearon usuarios desde el panel administrativo, se debe garantizar lo siguiente:

- Credenciales seguras.
- Roles específicos.
- Permisos asociados al cliente.

Para crear el usuario se debe primero ingresar al panel de Usuarios:





Después ingresar las configuraciones y datos básicos del usuario

Por último, asignarle una contraseña segura al usuario

7. Configuración del Cliente

El cliente es el componente que representa la aplicación frontend dentro de Keycloak.

7.1 Tipo de Cliente

Se configuró como:

- **Public Client**
- Sin autenticación por client secret.



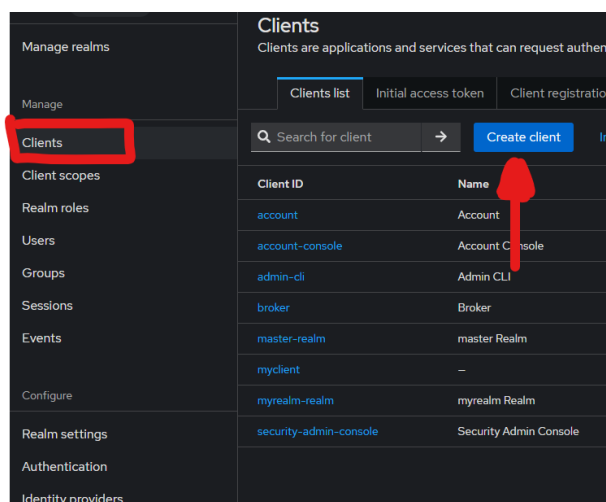
UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

Vicerrectoría
Oficina de Extensión

Académica

Esto se debe a que las aplicaciones frontend no pueden almacenar secretos de forma segura.

En primera instancia se debe ingresar al panel de clientes y se crea un nuevo cliente



Luego se configura el cliente teniendo en cuenta que el ClientID es importante debido a que es el ID que se utilizará para lograr la conexión desde los aplicativos hacia el SSO

Create client
Clients are applications and services that can request authentication of a user.

1 General settings

Client type ⓘ OpenID Connect

Client ID * ⓘ clientID

Name ⓘ

Description ⓘ

Always display in UI ⓘ ☐ Off

Por último, se configura el tipo de cliente y otros parámetros del mismo



8. Configuración de Seguridad del Cliente

8.1 Client Authentication

- Desactivado para frontend.
- Evita errores 401.
- Cumple buenas prácticas para SPAs.

8.2 Authorization Services

- Desactivado.
- No requerido para control básico por roles.
- Reduce complejidad innecesaria.

9. Flujos de Autenticación (OAuth 2.0 / OIDC)

9.1 Standard Flow (Authorization Code Flow)

- Habilitado
- Compatible con PKCE
- Recomendado para SPAs



Este flujo:

- Redirige al usuario al login.
- Devuelve un authorization code.
- Intercambia el código por tokens.

Es el flujo recomendado por estándares modernos.

9.2 Implicit Flow

- Deshabilitado

Razones:

- Tokens expuestos en URL.
- Vulnerable a XSS.
- Obsoleto en OAuth 2.1.

9.3 Direct Access Grants

- Deshabilitado para frontend
- Se utiliza únicamente en scripts o automatizaciones controladas.

9.4 Service Account Roles

- No habilitado
- Solo aplica para comunicación máquina-a-máquina.

9.5 Token Exchange, Device Flow y CIBA

- No aplicables para SPA.
- No requeridos para este proyecto.

10. PKCE (Proof Key for Code Exchange)

Se configuró:

- Método S256 (obligatorio)



PKCE protege contra:

- Interceptación del authorization code.
- Ataques de replay.

Es obligatorio en clientes públicos modernos.

11. DPOP Bound Tokens

- Deshabilitado

Razones:

- Tecnología reciente.
- No soportado por keycloak-js.
- No requerido para el alcance actual.

12. Gestión de Roles

Se configuraron roles desde:

- Panel de Roles del Realm.

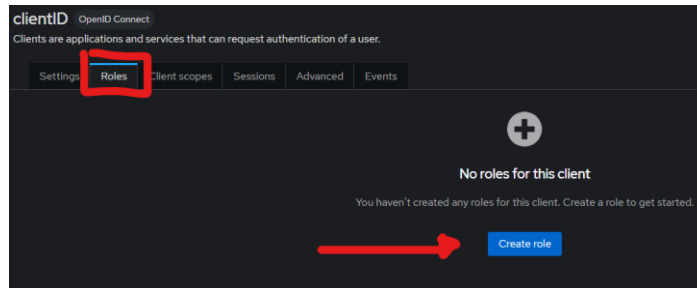


UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

Vicerrectoría
Oficina de Extensión

Académica

- Asignación al cliente.



- Asociación a usuarios.

La aplicación consulta los roles desde el token JWT para:

- Control de acceso.
- Renderizado condicional de componentes.
- Autorización lógica en frontend.



13. Implementación en Frontend

Se implementó middleware utilizando la librería `keycloak-js`.

Funcionalidades implementadas:

- Inicialización obligatoria con `login-required`.
- Validación de autenticación.
- Obtención de roles.
- Verificación de roles específicos.
- Obtención de usuario autenticado.

El middleware permite:

- Protección global de rutas.
- Control por roles.
- Validación de sesión activa.
- Acceso dinámico según permisos.

14. Seguridad Implementada

La configuración aplicada garantiza:

- Uso obligatorio de HTTPS.
- Flujo Authorization Code con PKCE.
- Eliminación de flujos inseguros.
- Cliente público sin exposición de secretos.
- Tokens gestionados bajo estándar OpenID Connect.
- Protección ante interceptación de código.



15. Resultados

Se logró:

- Integración exitosa entre el sistema frontend y el SSO.
- Autenticación centralizada funcional.
- Control de acceso basado en roles.
- Arquitectura compatible con microservicios.
- Cumplimiento de buenas prácticas de seguridad.

16. Conclusiones

La implementación del sistema SSO con Keycloak para aplicaciones frontend fue realizada correctamente bajo estándares modernos de seguridad y arquitectura.

El uso de:

- Authorization Code Flow
- PKCE (S256)
- Cliente público
- Proxy seguro con HTTPS

garantiza un entorno robusto, escalable y alineado con buenas prácticas internacionales.

La solución es estable, segura y apta para producción.